COMP-251 Answer Key

Course: COMP-251 **Total Marks: 40**

Section A: Graph Algorithms

Question 1 (10 marks) - graphs

Given a directed weighted graph G = (V, E) with non-negative edge weights and a set of k designated hub vertices, design an algorithm to compute the shortest path from source s to target t that passes through at least one hub vertex. State its worst-case time and space complexity.

> ***Correct Solution:*** *Run Dijkstra's algorithm forward from s on G and backward from t on the reversed graph G^R. Take the minimum over all hubs h in H of dist(s, h) + dist(h, t).*
> ***Explanation:*** *Two single-source shortest-path runs take O((\|V\| + \|E\|) log \|V\|) time using a binary heap, avoiding running Dijkstra from every hub.*

Question 2 (10 marks) - dynamic_programming

A data stream arrives in n intervals with processing costs c[1..n] and cooldown step k. Formulate a dynamic programming recurrence to maximize net throughput without scheduling two intervals within k steps of each other.

> ***Correct Solution:*** *Let DP[i] = max(DP[i - 1], gain[i] + DP[max(0, i - k - 1)]) for i = 1..n with base case DP[0] = 0.*
> ***Explanation:*** *Optimal substructure holds because choosing interval i restricts prior choices to indices <= i - k - 1. Time complexity is O(n) and space is O(n).*

Section A: Trees and Hashing

Question 1 (10 marks) - trees

Write a Python function `is_height_balanced(root)` that checks in O(n) time whether a binary tree is height-balanced, returning both the balance status and subtree height in a single post-order traversal.

> ***Correct Solution:*** *def check(node): if node is None: return True, 0 left_ok, left_h = check(node.left) right_ok, right_h = check(node.right) balanced = left_ok and right_ok and abs(left_h - right_h) <= 1 return balanced, 1 + max(left_h, right_h)*
> ***Explanation:*** *Combining height computation and balance verification in one bottom-up pass eliminates redundant subtree traversals and guarantees O(n) runtime.*

Section B: Dynamic Programming

Question 1 (10 marks) - hashing

Compare open addressing with linear probing against separate chaining when the load factor alpha approaches 0.9. State the expected probe count and cache locality trade-offs.

> ***Correct Solution:*** *Linear probing suffers primary clustering as alpha -> 0.9, growing expected unsuccessful probe cost to O(1 / (1 - alpha)^2), whereas separate chaining degrades gracefully as O(1 + alpha) at the cost of pointer chasing.*
> ***Explanation:*** *Open addressing keeps entries contiguous in memory for cache efficiency at moderate load factors, but requires resizing before alpha becomes high.*